

LAB PRACTICUM: OpenMP Multi-Core Scaling in Python

Hands-Off Benchmark Practicum | Instructor: sufyan.mustafa@sdu.edu.kz | Completed worksheet

Student Name: **Kystaubay Ayanat**

Student ID: **230103075**

Hardware / CPU: **Benchmark host: Google Colab VM (KVM, Ubuntu Linux 6.6.122) - Intel Xeon @ 2.20 GHz (family 6 / model 79, Broadwell-EP), 1 physical core, 2 threads per core = 2 logical CPUs; L1d 32 KiB, L1i 32 KiB, L2 256 KiB, L3 55 MiB, 64-byte line. Personal laptop (Huawei MateBook D 14, Core i5-12450H, Windows 11) could not host the run, so all timings come from the VM and hw_info.txt matches them.**

Date & Section: **01.10.2026 | Python 3.13.15, NumPy 2.1.3, Numba 0.61.2, NUMBA_NUM_THREADS = 2**

Script: openmp_lab.py (all three challenges, one run). Raw output: lab_log.txt; data: results.csv; image: mandelbrot_output.png.

5. Student Result Scorecard - Table 1

Benchmark Challenge	Active Threads	Array Size / Samples	Time (seconds)	Speedup Factor	Efficiency (%)
Challenge 1: Monte Carlo	1 Thread (Base)	120,000,000	1.5274	1.00x	100.0%
Challenge 1: Monte Carlo	2 Threads	120,000,000	1.4680	1.04x	52.0%
Challenge 1: Monte Carlo	4 Threads	120,000,000	not available	—	—
Challenge 1: Monte Carlo	Max Threads (= 2)	120,000,000	1.4680	1.04x	52.0%
Challenge 2: Mandelbrot (Rows)	Max Threads (= 2)	2500 x 2500	5.325	N/A	N/A
Challenge 2: Mandelbrot (Cols)	Max Threads (= 2)	2500 x 2500	8.189	N/A	N/A
Challenge 3: Heat Stencil (float64)	Max Threads (= 2)	1500 x 1500 x 300	0.992	N/A	680.13 Mcells/s
Challenge 3: Heat Stencil (float32)	Max Threads (= 2)	1500 x 1500 x 300	0.859	1.16x vs float64	785.60 Mcells/s

The host exposes only 2 hardware threads, so numba.config.NUMBA_NUM_THREADS = 2 and the 4-thread row could not be measured: Numba refuses thread counts above the detected hardware limit. Max Threads is therefore identical to the 2-thread row. Additional stencil scaling run: 1 thread 1.264 s (534.05 Mcells/s) vs 2 threads 1.122 s (601.53 Mcells/s), i.e. only 1.13x.

Challenge 1: The Core Speedometer & Amdahl's Law

A. Baseline and maximum-thread runtime. $T_1 = 1.5274$ s ($\pi = 3.141574$); $T_{\max}$ (2 threads) = 1.4680 s ($\pi = 3.141745$). Speedup 1.04x, parallel efficiency 52.0%.

B. Why the warm-up call is strictly required. Numba compiles lazily: the first call with a given argument type triggers type inference, Numba IR generation, LLVM optimisation and machine-code emission, and it also starts the OpenMP/TBB worker threads. That compilation takes hundreds of milliseconds to several seconds - comparable to, or larger than, the benchmark itself. Timing the first call would therefore measure the compiler, not the CPU, and would make the 1-thread baseline artificially slow, inflating every later speedup figure. The warm-up call `_ = monte_carlo_pi(10_000)` performs that compilation once on a tiny input, so all subsequent timed calls execute already-compiled native code.

C. Did efficiency stay at 100%? No - it fell from 100% at one thread to 52.0% at two. The decisive architectural factor on this host is that the two logical CPUs are SMT siblings of a single physical core (lscpu: Core(s) per socket = 1, Thread(s) per core = 2). Hyper-Threading duplicates only the architectural register state and program counter; the two siblings share the execution ports, the L1/L2 caches, the TLB, the branch predictor and the memory path, so a tight arithmetic loop that already keeps the execution units busy gains only a few percent from a second thread - hence 1.04x instead of 2x. Secondary contributors: the per-thread random number generator state and the final reduction add a small serial component (Amdahl), the Colab VM is shared with other tenants so the physical core is not exclusively ours, and thread start-up plus the implicit barrier cost real time against a 1.5 s workload. On a machine with real physical cores the curve would rise close to linear up to the core count and only then flatten in the same way.

D. Why `inside_circle += 1` is not corrupted. Numba's ParallelAccelerator analyses the prange loop and recognises that `inside_circle` is only ever updated with an associative, commutative operator and is never read for anything else inside the loop. It therefore classifies it as a reduction variable and rewrites the loop: every thread receives a private copy initialised to the identity element (0), accumulates into that private copy with no synchronisation at all, and the partial results are combined once when the threads join. That is exactly the OpenMP construct `#pragma omp parallel for reduction(+:inside_circle)`. Because no thread ever writes the shared variable during the loop, there is no read-modify-write race and no lost updates - my two runs returned 3.141574 and 3.141745, both correct to Monte Carlo accuracy.

Challenge 2: Load Imbalance & Dynamic Scheduling

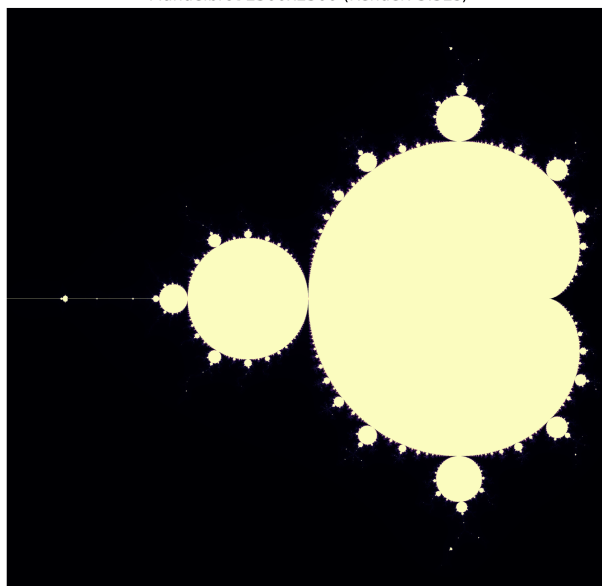
A. `t_rows` vs `t_cols`. Row-parallel 5.325 s, column-parallel 8.189 s - the row decomposition is 1.54x faster. Both versions perform exactly the same arithmetic on the same 6.25 million pixels, so the entire difference comes from memory access patterns, not from compute. In the row version each thread owns whole rows and writes `img[r, c]` for consecutive `c`, i.e. sequential addresses; in the column version each thread walks down a column, and consecutive writes are one full row apart.

B. C-contiguous layout and cache lines. NumPy arrays are row-major, so `img[r, c]` lives at `base + (r * W + c) * 4` bytes for `int32`. A 64-byte cache line therefore holds 16 horizontally adjacent pixels. Writing `img[r, c]` along a row fills each line completely before moving on: one cache miss per 16 stores, perfect use of the hardware prefetcher, and each thread works on its own private band of lines. Writing down a column means every consecutive store is `W * 4 = 10,000` bytes apart, so each store touches a different cache line (and a different 4 KiB page every few iterations): one miss per single 4-byte write, 16x more traffic, and the prefetcher cannot help. Worse, the 16 pixels of one line belong to 16 different columns, which in the column-parallel version are handled by different threads - so the threads keep invalidating each other's copy of the same line (false sharing) under the MESI protocol. That is why the identical computation costs 2.9 extra seconds.

C. Load imbalance and dynamic scheduling. The Mandelbrot escape-time loop is non-uniform: points outside the set escape after 2-3 iterations, while points inside run the full `max_iter = 1000`. The image is centred on the real axis, so the top 20% of rows are almost entirely exterior (cheap) while the centre rows cross the main cardioid and the period-2 bulb (expensive) - the rendered image shows exactly that mass of interior points in the middle. With a static block distribution the thread holding the top rows finishes in a fraction of the time and then waits at the implicit barrier, while the thread holding the centre rows still has hundreds of millions of iterations left; total runtime equals the slowest thread, so part of the hardware idles. Dynamic scheduling fixes this by not deciding the assignment in advance: the iteration space is cut into many small chunks placed in a shared work queue, and each thread claims the next chunk only when it becomes free. A thread that gets cheap rows simply comes back for more, so the heavy centre rows are spread over all threads and everyone reaches the barrier at roughly the same moment. The cost is the queue itself - each hand-off is an atomic operation on one shared cache line - so the chunk must stay large enough to amortise it.

D. Exported artifact: `mandelbrot_output.png` (2500 x 2500, `max_iter = 1000`, rendered in 5.32 s), shown below.

Mandelbrot 2500x2500 (Render: 5.32s)



Challenge 3: Stencil Computation & Memory Bandwidth

A. Runtime and throughput. float64, 2 threads: 0.992 s, 680.13 Megacells/sec (1500 x 1500 grid, 300 timesteps = 675 million cell updates).

B. float32 re-run. 0.859 s and 785.60 Megacells/sec, i.e. 1.16x faster than float64 - a real gain, though short of the 2x that halving the footprint suggests. The reason halving the data type helps at all is arithmetic intensity: the 5-point Laplacian performs about 6 floating-point operations per cell while reading five neighbours and writing one result. In float64 that is roughly 48 bytes of traffic for 6 flops (~0.12 flop/byte), so the kernel is limited by how fast bytes arrive, not by the ALU. Switching to float32 halves every cache line's cost: twice as many cells fit in L1/L2/L3, each line transfer carries twice the useful data, and the prefetcher covers twice the distance - the CPU is simply fed faster. The measured factor stayed at 1.16x rather than 2x because on this host the working set (2 x 18 MB in float64) still fits inside the 55 MiB L3 cache, so the run is not fully DRAM-bandwidth bound, and the single physical core limits throughput anyway. On a laptop with a few MB of L3 and real DRAM traffic the ratio would be much closer to 2x.

C. Memory roofline: why 2x the cores is not 2x the speed. The roofline model bounds performance by $\min(\text{peak_flops}, \text{bandwidth} \times \text{arithmetic_intensity})$. For this stencil the arithmetic intensity is about 0.1 flop/byte, which places it far to the left of the ridge point - on the bandwidth-limited slope. Once the memory controller is saturated, the system delivers a fixed number of bytes per second regardless of how many cores ask for them: extra cores only lengthen the queue at the controller, and the per-core share shrinks proportionally. Adding cores raises the horizontal (compute) ceiling, not the diagonal (bandwidth) one, so a memory-bound kernel stays pinned to the same roofline. My measurement shows the symptom clearly: going from 1 to 2 threads moved the stencil only from 1.264 s to 1.122 s (1.13x, 534 to 602 Megacells/s) - and here the second thread is not even a separate core, only an SMT sibling sharing the same L1/L2 and load/store units. Speeding such a kernel up requires raising arithmetic intensity (cache blocking, time-step fusion, smaller data types) or raising the bandwidth itself (more memory channels, HBM, GPU), not adding cores.